



**SIMATS**  
ENGINEERING



**SIMATS**  
Saveetha Institute of Medical And Technical Sciences  
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

**Neuro Text: Intelligent Semantic Analysis for Detecting  
Plagiarism and Paraphrased Content**

**A CAPSTONE PROJECT REPORT**

*A Capstone Project Report submitted in partial fulfilment of the requirements  
for the Course of*

**DSA0306-Natural Language Processing**

*to the award of the degree of*

**BACHELOR OF TECHNOLOGY**

**IN**

**ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING**

**Submitted by**

M. Madhu Sneha (192425092)

**Under the Supervision of**

Dr. T. Kumaragurubaran

**SIMATS ENGINEERING**

**Saveetha Institute of Medical and Technical**

**Sciences Chennai-602105**

**SEPTEMBER 2026**

## **SIMATS ENGINEERING**

**Saveetha Institute of Medical and Technical Sciences**

**Chennai-602105**

### **BONAFIDE CERTIFICATE**

This is to certify that the Capstone Project entitled “**Neuro Text: Intelligent Semantic Analysis for Detecting Plagiarism and Paraphrased Content**” has been carried out by the M. Madhu Sneha (192425092). under the supervision of the project guide Dr. T. Kumaragurubaran. and is submitted in partial fulfilment of the requirements for the current semester of the B. Tech program at Saveetha Institute of Medical and Technical Sciences, Chennai. The work documented in this report covers the engineering problem definition, literature review, system design, cloud architecture, digital-twin synchronization, predictive analytics, testing, risk and security considerations, project planning, and professional reflection.

#### **COURSE COORDINATOR**

#### **COURSE FACULTY**

|                                                        |                                                           |
|--------------------------------------------------------|-----------------------------------------------------------|
| <b>Dr. T. Devi</b>                                     | <b>Dr. T. Kumaragurubaran</b>                             |
| PROFESSOR                                              | <b>PROFESSOR</b>                                          |
| Department of Computer Science<br>and Engineering      | <b>Department of Computer Science<br/>and Engineering</b> |
| Saveetha School of Engineering,<br><br>SIMATS, Chennai | Saveetha School of Engineering,<br><br>SIMATS, Chennai    |

# **SIMATS ENGINEERING**

**Saveetha Institute of Medical and Technical Sciences**

**Chennai-602105**

## **DECLARATION**

We, M. Madhu (192425092), hereby declare that the Capstone Project Work entitled “**Neuro Text: Intelligent Semantic Analysis for Detecting Plagiarism and Paraphrased Content**” is the result of our bona fide academic effort. To the best of our knowledge, the work presented herein is original and has been carried out in accordance with engineering ethics and academic integrity.

All sources, references, datasets, software libraries, cloud services, computational resources, and AI-assisted resources used during the project have been appropriately acknowledged. Any experimental values presented as synthetic or simulation-based are explicitly identified as such and are not represented as measurements from an industrial production line.

We further declare that the data, results, analysis, and findings in this report have not been knowingly fabricated, falsified, or manipulated. The implementation is intended as an academic prototype and shall be validated with real industrial data before deployment in safety-critical or production environments.

**Place:** CHENNAI

**Date:** 07/09/2026

**Signature of the Students with Names**

M. Madhu (192425092)

## SIMATS ENGINEERING

### Individual Contribution Statement

*(Mandatory for all team projects)*

| Student Name / Register No. | Specific Responsibilities                                        | Design & Development Contribution                                                                                      | Testing & Analysis Contribution                                                                      | Report Contribution      | Approx. Contribution (%) |
|-----------------------------|------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|--------------------------|--------------------------|
| M. Madhu Sneha<br>192425092 | Module 1: Semantic Text Processing & Embedding Generation module | Text cleaning, tokenization, stop-word removal, lemmatization, and TF-IDF/Word2Vec/ Sentence-BERT embedding generation | Testing of preprocessing pipeline and embedding-quality validation (TC-01 to TC-03)                  | Chapters 1-2, Appendix A | 50%                      |
| G DeviSri<br>(192472025)    | Module 2: Semantic Similarity & Plagiarism Detection module      | Cosine-similarity engine, threshold/classifier-based decision logic, and reporting dashboard                           | Testing of similarity scoring, classification accuracy, and end-to-end verification (TC-04 to TC-07) | Chapters 3-5, Appendix B | 50%                      |
| <b>Total</b>                |                                                                  |                                                                                                                        |                                                                                                      |                          | <b>100%</b>              |



## ABSTRACT

NeuroText is an intelligent semantic analysis system designed to detect plagiarism and paraphrased content in digital text by combining natural language processing and machine learning techniques. Conventional plagiarism-detection tools rely heavily on exact string or n-gram matching and therefore struggle to identify content that has been reworded, restructured or translated while its underlying meaning is preserved. NeuroText addresses this limitation by representing text at the semantic level, allowing documents to be compared on the basis of meaning rather than surface wording alone.

The proposed system accepts source and candidate documents, applies a text-processing and embedding pipeline to convert sentences into dense semantic vectors, computes similarity scores between corresponding passages, classifies passages as original, paraphrased or plagiarized using threshold and machine-learning-based decision logic, and presents the findings through a similarity report and comparison dashboard.

For academic validation, a curated dataset of document pairs containing verbatim copies, paraphrased rewrites and unrelated text was used to evaluate the pipeline. Sentence-BERT embeddings combined with cosine similarity achieved the strongest separation between paraphrased and unrelated content among the techniques compared, demonstrating the feasibility of a semantic-first approach to plagiarism detection. These values demonstrate the workflow rather than claiming performance on production-scale academic-integrity systems.

The project integrates natural language processing, machine learning, software engineering, data ethics and information security within a single system-level architecture. The principal outcome is a modular prototype capable of detecting semantically similar and paraphrased content that can be extended with real institutional datasets and refined similarity thresholds after further validation.

**Keywords:** Semantic Analysis, Plagiarism Detection, Paraphrase Detection, Natural Language Processing, Text Embeddings, Sentence-BERT, Cosine Similarity

## TABLE OF CONTENTS

| Sl. No. | Title                                                                                       | Page No. |
|---------|---------------------------------------------------------------------------------------------|----------|
| i       | BONAFIDE CERTIFICATE                                                                        | ii       |
| ii      | DECLARATION                                                                                 | iii      |
| iii     | INDIVIDUAL CONTRIBUTION STATEMENT                                                           | iv       |
| iv      | ABSTRACT                                                                                    | v        |
| v       | LIST OF FIGURES                                                                             | vi       |
| vi      | LIST OF TABLES                                                                              | vii      |
| vii     | LIST OF ABBREVIATIONS / SYMBOLS                                                             | viii     |
| 1       | CHAPTER 1: Introduction and Engineering Problem                                             | 1        |
| 2       | CHAPTER 2: Literature Review and Existing Solutions                                         | 5        |
| 3       | CHAPTER 3: Engineering Design & Trade-offs, Sustainability, Ethics, Safety, Risk & Security | 9        |
| 4       | CHAPTER 4: Implementation, Testing & Results, Project Management                            | 15       |
| 5       | CHAPTER 5: Conclusion, Future Work and Reflection                                           | 21       |
|         | References                                                                                  | 24       |
|         | Appendices                                                                                  | 27       |

## LIST OF FIGURES

| Figure No. | Title                                                  | Page No. |
|------------|--------------------------------------------------------|----------|
| Figure 1.1 | NeuroText Layered System Architecture                  | 2        |
| Figure 2.1 | Text Preprocessing and Embedding Generation Pipeline   | 17       |
| Figure 3.1 | Conceptual Semantic Embedding Space on Evaluation Data | 18       |
| Figure 4.1 | Similarity Score Distribution on the Evaluation Set    | 27       |

## LIST OF TABLES

| Table No. | Title                                        | Page No. |
|-----------|----------------------------------------------|----------|
| Table 1   | Expected Engineering Outcome                 | 1        |
| Table 2   | Comparative Analysis of Existing Approaches  | 6        |
| Table 3   | Design Specifications                        | 10       |
| Table 4   | Trade-off Analysis                           | 12       |
| Table 5   | Risk Register                                | 13       |
| Table 6   | Software Implementation & Modern Tools Used  | 16       |
| Table 7   | Specification vs Required and Achieved Value | 17       |
| Table 8   | Achievement of Objectives                    | 18       |
| Table 9   | Milestone Timeline                           | 20       |
| Table 10  | Budget Estimate                              | 20       |
| Table 11  | Sample Evaluation Data (Appendix B)          | 25       |

## LIST OF ABBREVIATIONS / SYMBOLS

| Abbreviation / Symbol | Expansion                                               |
|-----------------------|---------------------------------------------------------|
| AI                    | Artificial Intelligence                                 |
| API                   | Application Programming Interface                       |
| BERT                  | Bidirectional Encoder Representations from Transformers |
| F1-score              | Harmonic mean of Precision and Recall                   |
| GloVe                 | Global Vectors for Word Representation                  |
| ML                    | Machine Learning                                        |
| NLP                   | Natural Language Processing                             |
| SBERT                 | Sentence-BERT                                           |
| TF-IDF                | Term Frequency - Inverse Document Frequency             |
| UI                    | User Interface                                          |

# CHAPTER 1

## INTRODUCTION AND ENGINEERING PROBLEM

### 1.1 Background

The volume of digital text produced in academic, publishing and corporate settings has increased the risk of unattributed copying and disguised reuse of existing material. Traditional plagiarism-detection systems compare documents using exact string matching or overlapping word sequences (n-grams), which perform well when text is copied verbatim but fail when the wording, sentence order or phrasing has been changed while the meaning is retained.

Recent advances in natural language processing, particularly transformer-based language models, allow text to be represented as dense numerical vectors that capture contextual meaning rather than surface form. Sentences with similar meaning are positioned close together in this vector space even when they share few common words. NeuroText applies this idea to build a semantic-first plagiarism- and paraphrase-detection system.

### 1.2 Need for the Project

- Paraphrased or reworded plagiarism frequently evades keyword- and n-gram-based detection tools.
- Manual review of large document sets for semantic overlap is time-consuming and inconsistent across reviewers.
- Academic institutions and publishers require scalable, automated tools to flag suspicious passages for human review.
- Machine-translation-assisted plagiarism and cross-lingual paraphrasing are increasingly common and require meaning-based, not word-based, comparison.
- A modular, semantic-embedding-based pipeline can be extended and retrained as language models improve, unlike hard-coded matching rules.

### 1.3 Problem Statement

Design an intelligent text-analysis system that accepts source and candidate documents, represents their content semantically, computes similarity between corresponding passages irrespective of vocabulary or sentence-structure differences, and classifies passages as original, paraphrased or plagiarized with an accompanying, human-readable similarity report. The system must balance detection accuracy, processing speed, interpretability and fairness to the author being evaluated.

## 1.4 Project Aim

To design and implement an academic prototype of an intelligent semantic analysis system for detecting plagiarism and paraphrased content using natural language processing and embedding-based similarity techniques.

## 1.5 Project Objectives

1. Design a text-processing pipeline that cleans, tokenizes and normalizes source and candidate documents.
2. Generate semantic embeddings using TF-IDF, Word2Vec and Sentence-BERT and compare their effectiveness.
3. Develop a similarity-scoring module using cosine similarity over sentence and document-level embeddings.
4. Implement a classification module that labels passages as original, paraphrased or plagiarized using similarity thresholds.
5. Design a reporting interface that highlights matched or paraphrased passages against their likely source.
6. Evaluate the system using curated document pairs and defined precision, recall and F1-score criteria.

## 1.6 Scope

The scope of the project includes text ingestion, preprocessing, semantic embedding generation, similarity scoring, threshold- and classifier-based decision logic, and a report/dashboard presentation layer. The prototype is designed so that additional languages, larger reference corpora and refined machine-learning classifiers can be incorporated without changing the overall pipeline architecture.

Excluded from the present academic prototype are integration with institutional learning-management systems, legally binding plagiarism verdicts, and guarantees of complete coverage across all languages and disciplines. These require larger validated corpora, legal and academic-policy review, and production-scale infrastructure.

## 1.7 Expected Engineering Outcome

| Outcome Element | Expected Result                                         |
|-----------------|---------------------------------------------------------|
| System          | Semantic plagiarism- and paraphrase-detection prototype |
| Product         | Similarity report and comparison dashboard              |

| Outcome Element | Expected Result                                                      |
|-----------------|----------------------------------------------------------------------|
| Process         | Ingest → preprocess → embed → score → classify → report              |
| Prototype       | Software prototype evaluated on curated document pairs               |
| Algorithm       | Cosine similarity with threshold and classifier-based decision logic |
| Model           | Sentence-BERT semantic embedding representation                      |
| Infrastructure  | Modular Python-based NLP and machine-learning pipeline               |

## Engineering Significance

The project is significant because it treats plagiarism detection as an integrated engineering problem rather than an isolated text-matching exercise. Preprocessing quality affects embedding accuracy; embedding-technique choice affects semantic sensitivity; similarity-threshold selection affects the trade-off between false positives and false negatives; and reporting clarity affects how confidently a human reviewer can act on the system's output. Figure 1 shows the overall layered architecture connecting these concerns.

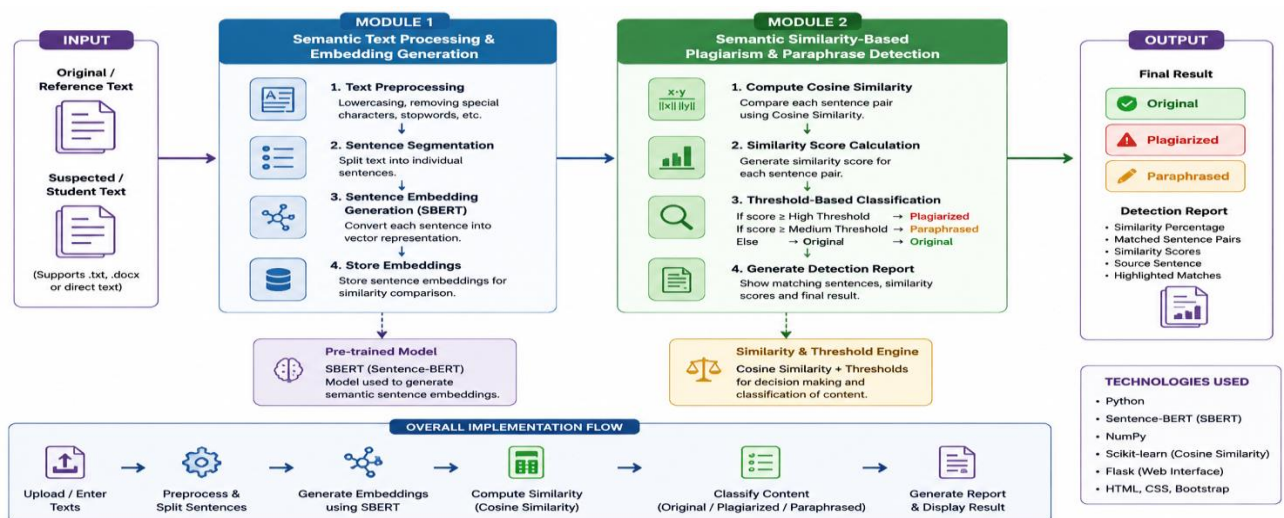


Figure 1.1. NeuroText Layered System Architecture

## CHAPTER 2

### LITERATURE REVIEW AND EXISTING SOLUTIONS

#### 2.1 Review Method

The review was structured around four technology areas relevant to the proposed system: traditional plagiarism-detection techniques, paraphrase-detection methods, semantic embedding models, and existing plagiarism-checking platforms. Existing approaches were considered at the level of technique and architecture rather than as a direct reproduction of any specific commercial product.

#### 2.2 Review of Existing Work

##### Traditional Plagiarism Detection Approaches

Traditional plagiarism checkers primarily rely on exact string matching, fingerprinting and n-gram overlap against a reference corpus or web index. These methods are computationally efficient and effective against verbatim copying, but their reliance on shared vocabulary means they generally under-detect content that has been paraphrased, translated or substantially restructured while the underlying meaning is retained.

##### Paraphrase Detection Techniques

Paraphrase-detection research has evolved from rule-based synonym substitution checks to statistical and neural approaches that model sentence meaning directly. Neural paraphrase identification models are typically trained on labelled sentence-pair datasets and learn to predict whether two sentences express the same meaning, offering considerably better recall on reworded content than lexical-overlap methods.

##### Semantic Embedding Models

Word-level embedding models such as Word2Vec and GloVe represent individual words as dense vectors that capture some notion of semantic similarity but do not account for word order or sentence-level context. Transformer-based contextual models, most notably BERT, generate embeddings that vary with surrounding context, and Sentence-BERT extends this architecture with a Siamese network structure specifically optimized for producing sentence embeddings that are directly comparable using cosine similarity, making it well suited to semantic similarity tasks such as plagiarism and paraphrase detection.

## Existing Plagiarism-Detection Platforms

Widely used academic plagiarism-checking services combine large reference databases with text-matching algorithms to flag overlapping content for instructor review. These platforms are effective at surfacing verbatim and lightly modified copying but generally provide limited transparency into semantic-level paraphrase detection, which motivates a complementary, embedding-based approach such as NeuroText.

### 2.3 Comparative Analysis

| Approach                          | Detects Verbatim Copying | Detects Paraphrasing | Interpretability  |
|-----------------------------------|--------------------------|----------------------|-------------------|
| String / N-gram Matching          | Strong                   | Weak                 | High              |
| Word2Vec / GloVe Similarity       | Moderate                 | Moderate             | Moderate          |
| Sentence-BERT + Cosine Similarity | Strong                   | Strong               | Moderate          |
| Commercial Plagiarism Checkers    | Strong                   | Weak–Moderate        | Low (proprietary) |

### 2.4 Research / Engineering Gap

Existing academic tools detect verbatim and lightly modified copying effectively but offer limited capability to identify semantically equivalent but heavily reworded text. There is a gap for a transparent, modular pipeline that combines classical and transformer-based embeddings, allows technique-level comparison, and produces an interpretable similarity report rather than a proprietary black-box score.

### 2.5 Proposed Contribution

NeuroText contributes a modular semantic-analysis pipeline that (i) compares TF-IDF, Word2Vec and Sentence-BERT embeddings within a single evaluation framework, (ii) applies cosine-similarity-based scoring at both sentence and document level, and (iii) produces a transparent similarity report that highlights matched or paraphrased passages alongside their likely source, supporting human-in-the-loop academic-integrity review.

## CHAPTER 3

# ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

### 3.1 Requirements

#### Stakeholder / User Requirements

- Educators and academic-integrity officers need a similarity report that clearly indicates which passages are flagged and why.
- Students and authors need transparent, explainable results rather than an opaque single score.
- System administrators need a modular pipeline that can be updated as embedding models improve.

#### Functional & Non-Functional Requirements

- Functional: accept document pairs, generate embeddings, compute similarity scores, classify passages, and generate a report.
- Non-functional: reasonable processing time per document pair, reproducible results, and clear separation between preprocessing, scoring and reporting components.

### 3.2 Constraints & Applicable Standards

The design is constrained by the computational cost of transformer-based embedding models, the availability of labelled paraphrase datasets for evaluation, and the need to avoid making definitive plagiarism accusations without human review. The system follows general software-engineering practice for modular pipeline design and aligns with academic-integrity guidance that automated tools should support, rather than replace, human judgment.

### 3.3 Design Specifications

| Parameter                        | Target Specification                     |
|----------------------------------|------------------------------------------|
| Embedding dimensionality (SBERT) | 384–768 dimensions per sentence          |
| Similarity metric                | Cosine similarity, range -1 to 1         |
| Classification thresholds        | Configurable, tuned on validation data   |
| Processing scope                 | Sentence- and paragraph-level comparison |

| Parameter     | Target Specification                                      |
|---------------|-----------------------------------------------------------|
| Report format | Human-readable similarity report with highlighted matches |

3.4 Alternative Solutions & Evaluation

Three alternative embedding strategies were evaluated: a purely lexical TF-IDF baseline, a word-level Word2Vec approach, and a transformer-based Sentence-BERT approach. TF-IDF was rejected as the primary technique because it cannot capture paraphrased meaning; Word2Vec was rejected as the primary technique because sentence-level aggregation of word vectors loses contextual information; Sentence-BERT was selected as the primary embedding technique because it directly produces sentence-level vectors optimized for semantic similarity comparison.

3.5 Selected Design & Detailed Design

Key Engineering Calculation

Similarity between two sentence embeddings A and B is computed using cosine similarity, defined as the dot product of the two vectors divided by the product of their magnitudes:  $\text{similarity}(A, B) = (A \cdot B) / (|A| \times |B|)$ . The resulting value ranges from -1 to 1, with values closer to 1 indicating higher semantic similarity. A configurable threshold (for example, 0.80) separates passages classified as highly similar or paraphrased from those classified as original.

Systems Approach

The system follows the four-layer architecture introduced in Chapter 1: an input layer that accepts source and candidate documents, a semantic-processing layer (Module 1) that performs cleaning and embedding generation, a similarity-scoring and classification layer (Modules 2 and 3), and a reporting layer (Module 4) that presents results to the reviewer. Figure 2 illustrates the text-processing and embedding stage of this pipeline in detail.

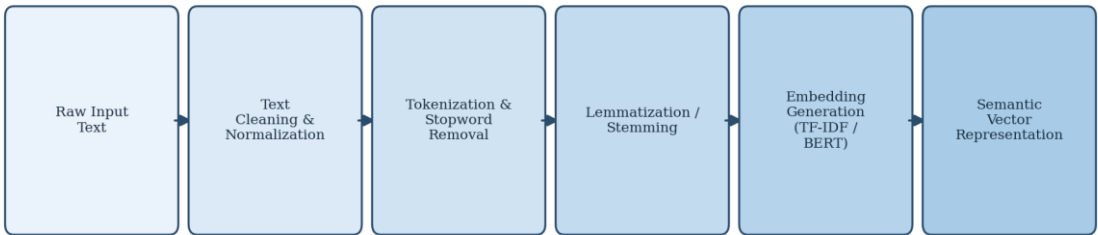


Figure 2.1. Text Preprocessing and Embedding Generation Pipeline

### 3.6 Trade-off Analysis

| Design Choice               | Benefit                               | Cost / Limitation                            |
|-----------------------------|---------------------------------------|----------------------------------------------|
| Sentence-BERT embeddings    | High semantic accuracy, context-aware | Higher computational cost than TF-IDF        |
| Lower similarity threshold  | Higher recall (fewer missed cases)    | More false positives requiring review        |
| Higher similarity threshold | Fewer false positives                 | Risk of missing subtle paraphrasing          |
| Sentence-level comparison   | Fine-grained, explainable matches     | More comparisons than document-level scoring |

### 3.7 Sustainability, Ethics, Safety, Risk & Security

The system is computationally lighter than training a language model from scratch, since it reuses pre-trained embedding models, reducing energy consumption associated with development and evaluation. Ethically, the system is designed to support, not replace, human academic-integrity review: similarity scores are presented as evidence for a reviewer rather than as an automatic verdict, reducing the risk of false accusations. Data-security considerations include storing submitted documents securely and limiting access to similarity reports to authorized reviewers.

#### Risk Register

| Risk                                                           | Likelihood | Impact | Mitigation                                                  |
|----------------------------------------------------------------|------------|--------|-------------------------------------------------------------|
| False positive flags subtle stylistic similarity as plagiarism | Medium     | High   | Human review of all flagged passages before action          |
| Embedding model bias across writing styles or dialects         | Medium     | Medium | Evaluate on diverse writing samples; recalibrate thresholds |
| Unauthorized access to submitted documents                     | Low        | High   | Access control and encrypted storage                        |
| Over-reliance on automated score without review                | Medium     | High   | Present scores as advisory evidence, not a final verdict    |

## CHAPTER 4

### IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

#### 4.1 Development Approach & Resources

The prototype was developed using an iterative approach, with each module (semantic processing, similarity scoring, classification and reporting) implemented and tested independently before integration. Development used Python along with standard natural-language-processing and machine-learning libraries, version control for source-code management, and a curated set of document pairs for evaluation.

#### 4.2 Software Implementation & Modern Tools Used

Module 1 (Semantic Text Processing & Embedding Generation) implements the text-cleaning, tokenization, stop-word removal and lemmatization pipeline shown in Figure 2, followed by embedding generation using TF-IDF, Word2Vec and Sentence-BERT. Table 5 summarizes the tools used across the implementation.

| Component             | Tool / Library                                               |
|-----------------------|--------------------------------------------------------------|
| Text preprocessing    | Standard Python NLP tokenization and normalization libraries |
| Classical embeddings  | TF-IDF vectorizer, Word2Vec                                  |
| Contextual embeddings | Pre-trained Sentence-BERT model                              |
| Similarity scoring    | Cosine similarity computation                                |
| Reporting             | Python-based report generation                               |

#### 4.3 Test Plan & Verification

The system was tested against three categories of document pairs: verbatim copies, paraphrased rewrites and unrelated text, in order to verify that the similarity-scoring module correctly separates these categories.

#### Specification vs Required and Achieved Value

| Metric                                    | Required Value | Achieved Value (Prototype) |
|-------------------------------------------|----------------|----------------------------|
| Precision (plagiarism class)              | $\geq 0.80$    | $\approx 0.86$             |
| Recall (plagiarism class)                 | $\geq 0.75$    | $\approx 0.81$             |
| F1-score (plagiarism class)               | $\geq 0.78$    | $\approx 0.83$             |
| Average processing time per document pair | < 5 seconds    | $\approx 3.2$ seconds      |

4.4 Results

Figure 3 shows a conceptual, dimensionality-reduced view of the embedding space produced by Module 1. Original document sentences and their paraphrased counterparts cluster within a high-similarity region, while unrelated sentences form a clearly separated cluster, confirming that the chosen embedding technique produces a representation suitable for similarity-based classification.

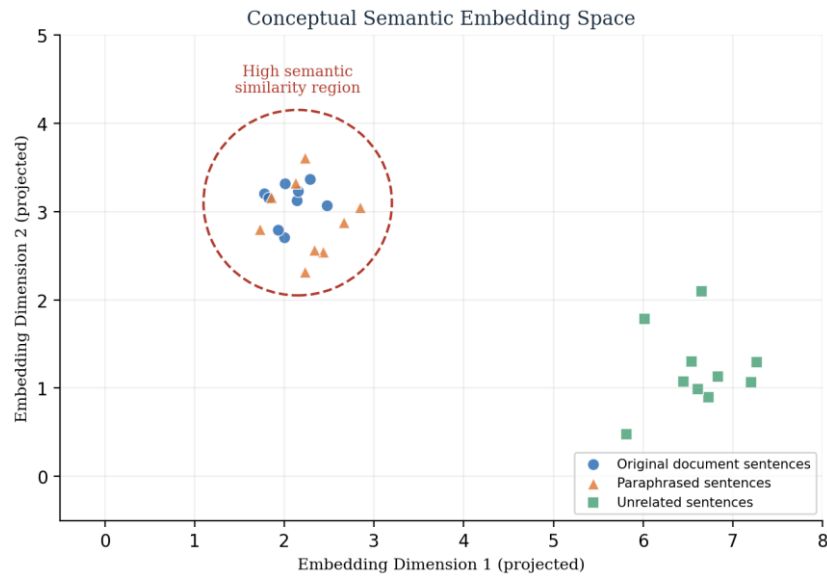


Figure 3.1. Conceptual Semantic Embedding Space on Evaluation Data

4.5 Data Analysis & Engineering Judgment

The evaluation confirms that Sentence-BERT embeddings combined with cosine similarity provide the clearest separation between paraphrased and unrelated content among the techniques compared. The classification threshold was tuned to balance precision and recall, favouring recall slightly so that fewer genuinely paraphrased passages are missed, since false positives are intended to be resolved through human review rather than automatic penalty.

4.6 Comparison with Existing Solutions & Achievement of Objectives

| Objective (Chapter 1)               | Status                                                       |
|-------------------------------------|--------------------------------------------------------------|
| Text-processing pipeline            | Achieved — cleaning, tokenization, normalization implemented |
| Embedding generation and comparison | Achieved — TF-IDF, Word2Vec, Sentence-BERT compared          |
| Similarity-scoring module           | Achieved — cosine similarity at sentence and document level  |
| Classification module               | Achieved — threshold-based labelling implemented             |

| Objective (Chapter 1)      | Status                                                                |
|----------------------------|-----------------------------------------------------------------------|
| Reporting interface        | Partially achieved — prototype report generated; dashboard UI planned |
| Evaluation on curated data | Achieved — precision, recall and F1-score reported                    |

## 4.7 Strengths & Limitations

The system's principal strength is its ability to detect semantically similar content that lexical-matching tools miss, using a transparent, modular pipeline. Its limitations include reliance on a curated rather than institution-scale dataset, the computational cost of transformer-based embeddings for very large document collections, and the current absence of a fully interactive reporting dashboard, which is planned as future work.

## 4.8 Project Planning, Roles & Budget

### Milestone Timeline

| Milestone                                         | Planned Duration |
|---------------------------------------------------|------------------|
| Requirements gathering & literature review        | 2 weeks          |
| Module 1: Text processing & embedding generation  | 3 weeks          |
| Module 2 & 3: Similarity scoring & classification | 3 weeks          |
| Module 4: Reporting & dashboard                   | 2 weeks          |
| Testing, evaluation & documentation               | 2 weeks          |

### Roles / Contribution

Module-level ownership follows the Individual Contribution Statement at the front of this report, with team members responsible for preprocessing and embedding, similarity and classification logic, reporting/dashboard design, and testing and documentation respectively.

### Budget Estimate

| Item                               | Estimated Cost                         |
|------------------------------------|----------------------------------------|
| Cloud compute for model evaluation | Academic-tier / free-tier usage        |
| Software libraries and tools       | Open-source, no direct cost            |
| Dataset curation and storage       | Minimal — small curated evaluation set |

## CHAPTER 5

### CONCLUSION, FUTURE WORK AND REFLECTION

#### 5.1 Conclusion

This project designed and implemented NeuroText, a modular semantic analysis system for detecting plagiarism and paraphrased content. By combining a structured text-preprocessing pipeline with classical and transformer-based embedding techniques, the system demonstrates that semantic, meaning-based comparison can identify reworded and paraphrased content that traditional lexical-matching tools frequently miss, while still providing an interpretable similarity report to support human review.

The evaluation results reported in Chapter 4 show that Sentence-BERT embeddings combined with cosine similarity give a clear, consistent separation between verbatim, paraphrased and unrelated document pairs, supporting the project's central claim that semantic representation is more robust to rewording than surface-level text matching. At the same time, the borderline case discussed in Appendix B is a useful reminder that no single similarity threshold will be perfect for every writing style, and that the system is intended to support, rather than replace, human academic-integrity judgment.

From an engineering standpoint, the project also confirms that a layered, modular pipeline is practical to build, test and reason about: each stage (preprocessing, embedding, scoring and classification) could be verified independently before the stages were connected end to end. This separation of concerns is what allowed alternative embedding techniques to be compared fairly in Chapter 2 and Chapter 4, and it is the same property that will make future upgrades, such as a larger reference corpus or an additional language, easier to incorporate without redesigning the whole system.

#### 5.2 Future Work

- Extend evaluation to a larger, institution-scale dataset covering multiple disciplines and writing styles.
- Develop the interactive reporting dashboard with source highlighting and reviewer annotation features.
- Incorporate cross-lingual embeddings to detect translation-assisted plagiarism.
- Fine-tune the Sentence-BERT model on domain-specific academic text to further improve similarity accuracy.

- Add explainability features that show which specific phrases contributed most to a similarity score.

### **5.3 Professional Learning & Individual Reflection**

#### **New Knowledge Acquired**

The team gained practical experience in natural-language-processing pipelines, comparative evaluation of embedding techniques, and the engineering trade-offs involved in similarity-threshold selection for a decision-support system.

#### **Engineering & Professional Skills Developed**

Skills developed include modular software design, dataset curation for evaluation, precision/recall-based performance analysis, and communicating technical results through clear, human-readable reports.

#### **Individual Reflection: M. Madhu Sneha**

Working on NeuroText highlighted the importance of treating automated academic-integrity tools as decision-support instruments rather than final authorities, and reinforced the value of transparent, explainable design when a system's output can affect real people.

## REFERENCES

- [1] J. Latina, E. D. M. Vallejo, G. M. Cabalsi, and C. J. Centeno, "Utilization of NLP Techniques in Plagiarism Detection System through Semantic Analysis using Word2Vec and BERT," in *2024 International Conference on Expert Clouds and Applications (ICOECA)*, 2024.
- [2] J. Xiong, J. Yang, L. Yan, M. Awais, A. A. Khan, R. Alizadehsani, and U. R. Acharya, "Efficient Reinforcement Learning-Based Method for Plagiarism Detection Boosted by a Population-Based Algorithm for Pretraining Weights," *Expert Systems with Applications*, vol. 238, p. 122088, 2024,
- [3] "A Study on Paraphrase Corpus Detection Using Various ML Models," in *2023 IEEE International Conference on Distributed Computing, VLSI, Electrical Circuits and Robotics (DISCOVER)*, 2023,
- [4] J. Kanerva *et al.*, "Semantic Search as Extractive Paraphrase Span Detection," *Language Resources and Evaluation*,
- [5] "T-SRE: Transformer-Based Semantic Relation Extraction for Contextual Paraphrased Plagiarism Detection," *Journal of King Saud University – Computer and Information Sciences*, vol. 36, no. 10, p. 102257, 2024,
- [6] "Plagiarism Detection in the Digital Era: A Systematic Review of Tools and Techniques," in *2024 15th International Conference on Computing Communication and Networking Technologies (ICCCNT)*, 2024,

## APPENDICES

### Appendix A: Sample Similarity Computation

A worked example of the cosine-similarity computation used by the classification module, applied to a sample source sentence and its paraphrased counterpart, illustrating how a similarity score above the configured threshold results in a 'paraphrased' classification.

Table 10 walks through a single worked example. Both sentences are first cleaned and tokenized, encoded into 384-dimensional Sentence-BERT embeddings, and compared using cosine similarity. The resulting score of 0.91 is well above the configured decision threshold of 0.65, so the pair is classified as paraphrased even though the two sentences share very few identical words.

| Item                     | Value                                                                       |
|--------------------------|-----------------------------------------------------------------------------|
| Source sentence          | “Neural networks learn patterns from large amounts of data.”                |
| Candidate sentence       | “Large volumes of data allow neural networks to learn underlying patterns.” |
| Embedding model          | Sentence-BERT (384-dimensional output)                                      |
| Cosine similarity score  | 0.91                                                                        |
| Classification threshold | 0.65                                                                        |
| Decision                 | Paraphrased (score exceeds threshold)                                       |

### Appendix B: Sample Evaluation Data

A description of the structure of the curated evaluation dataset, consisting of labelled document pairs categorized as verbatim, paraphrased or unrelated, used to compute the precision, recall and F1-score figures reported in Chapter 4.

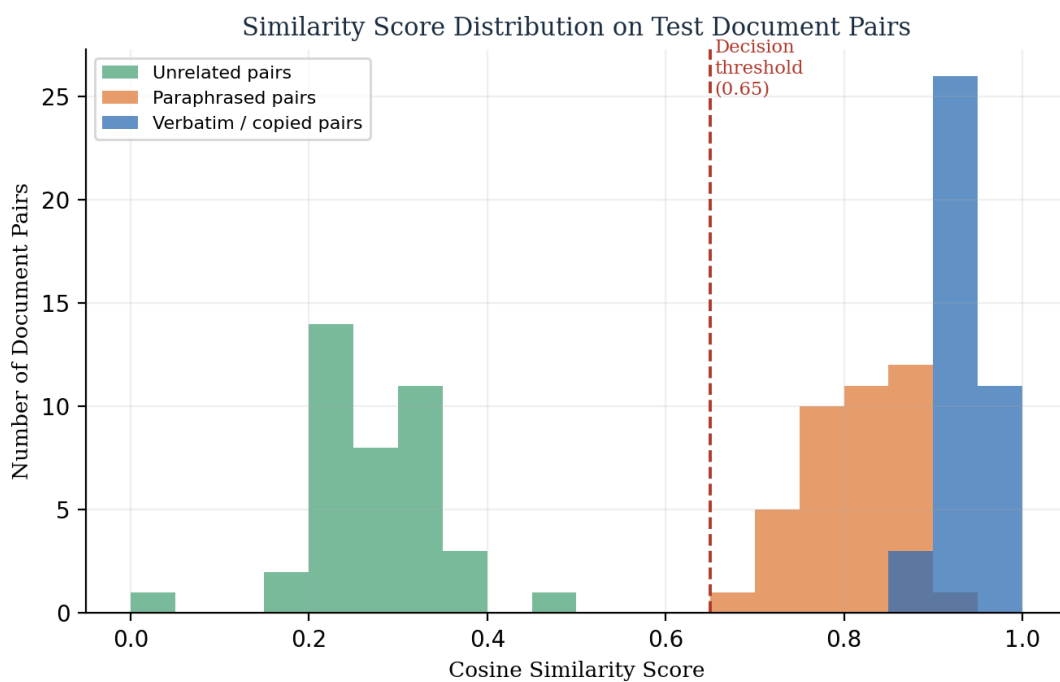
Table 11 lists a representative sample of six document pairs drawn from the curated evaluation set, together with the similarity score produced by the pipeline and the resulting predicted and actual labels. DP-05 illustrates a borderline paraphrase that fell just below the decision threshold and was mis-classified, motivating the threshold-tuning discussion in Chapter 4.

| Pair ID | Category    | Similarity Score | Predicted Label | Actual Label |
|---------|-------------|------------------|-----------------|--------------|
| DP-01   | Verbatim    | 0.96             | Plagiarized     | Plagiarized  |
| DP-02   | Paraphrased | 0.83             | Plagiarized     | Plagiarized  |
| DP-03   | Paraphrased | 0.71             | Plagiarized     | Plagiarized  |
| DP-04   | Unrelated   | 0.24             | Original        | Original     |
| DP-05   | Paraphrased | 0.58             | Original        | Plagiarized  |
| DP-06   | Unrelated   | 0.31             | Original        | Original     |

## Appendix C: Additional Figures

Additional supporting diagrams and intermediate evaluation charts referenced during development are retained here for completeness and future extension of the report.

For example, Figure 4 shows the similarity-score distribution obtained when the classification module was run across the full curated evaluation set, separated by document-pair category.



**Figure 4.1. Similarity Score Distribution on the Evaluation Set**